

La estrategia es profesional, dividir el entorno en dos carpetas independientes (`Clase_WEBXR` y `ClaseWebXRMeta`) es la mejor decisión técnica para mantener el orden y la estabilidad en el salón de clases.

Intentar usar un solo servidor que mezcle HTTP y HTTPS de forma local suele generar conflictos de rutas, problemas con la caché de los navegadores y pérdida de tiempo frente a los alumnos. Con este enfoque híbrido, se tiene el control absoluto de la clase.

Aquí un análisis del por qué esa idea es la correcta y cómo asegurar que convivan a la perfección:

Comparativa de tus dos entornos locales

Característica	Entorno Celulares (<code>Clase_WEBXR</code>)	Entorno Meta Quest (<code>ClaseWebXRMeta</code>)
Carpeta en D:	<code>Clase_WEBXR</code>	<code>ClaseWebXRMeta</code>
Protocolo	HTTP nativo (vía IP <code>192.168.0.100</code>)	HTTPS forzado (vía certificado SAN)
Comando CMD	<code>uvicorn main:app --host 0.0.0.0 --port 8000</code>	<code>python main.py</code> <i>(ejecutar el script autónomo)</i>
Puerto recomendado	<code>8000</code>	<code>8443</code> <i>(Cambio sugerido para evitar colisiones)</i>
Acceso en	<code>http://</code>	<code>https://la-</code>

dispositivo	192.168.0.100:8000	direccion-fija.local:8443
Configuración inicial	chrome://flags en cada teléfono	Aceptar advertencia "Avanzado / Proceder" en el visor

3 Recomendaciones clave para que el sistema funcione

1. Cambiar el puerto en la versión de Meta (`ClaseWebXRMeta`):
En el archivo `main.py` de la carpeta Meta, cambiar la variable a `PUERTO_SERVICE = 8443`. Al usar el puerto 8000 para HTTP y el 8443 para HTTPS, se podrá tener ambos servidores encendidos al mismo tiempo en la Alienware si así se desea, sin que se bloqueen entre sí.
2. Estructura interna idéntica:
Como los códigos HTML y los objetos 3D de Three.js son los mismos, solo hay que asegurarse de que en `ClaseWebXRMeta` el script de Python apunte correctamente a la carpeta donde se guardas el proyecto (por ejemplo, si se `StaticFiles(directory=".", ...)` como en los teléfonos, adptarlo en el script de Meta para que lea la misma raíz).
3. El fin definitivo de los "dientes de sierra" y el lag:
Al ejecutar el proyecto de forma inalámbrica en `ClaseWebXRMeta`, el visor Quest 3 usará su propio procesador gráfico (Snapdragon XR2 Gen 2) para renderizar la escena de Three.js. Al no usar el cable Link, la tarjeta integrada

de la Alienware ya no causará interferencias y la propiedad `antialias: true` se encargará de dejar los bordes perfectamente nítidos.

Guía Técnica: Servidor Local HTTPS Autónomo para Meta Quest 3

Esta guía te permitirá desplegar escenas de Three.js y WebXR de manera 100% local, inalámbrica y sin depender de una cuenta de desarrollador de Meta. Utiliza el procesador nativo del visor para eliminar conflictos gráficos, caídas de rendimiento y los dientes de sierra en la laptop Alienware.

Paso 1: Configurar el Entorno en tu Alienware

Para que el servidor de Meta funcione, se necesita instalar las librerías encargadas de la criptografía (para generar los certificados de seguridad SSL sobre la marcha) junto con la estructura de FastAPI.

1. Abrir la terminal de comandos (CMD o PowerShell) en la Alienware.
2. Ejecutar el siguiente comando para instalar los paquetes necesarios:

```
pip install cryptography uvicorn fastapi
```

Paso 2: El Código Completo de Python (Servidor HTTPS Autónomo)

Crear un archivo llamado `main.py` dentro de la nueva carpeta `D:\Diseno Interactivo 2 Sem 2025\ClaseWebXRMeta\`.

Este código unifica la lógica de generación automática de certificados SSL con soporte SAN (evitando el bloqueo de Chromium) y monta la raíz del servidor exactamente igual a como trabaja el entorno de teléfonos, pero utilizando el puerto seguro 8443.

```
import datetime
import os
import socket
import ipaddress
import uvicorn
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.x509.oid import NameID
```

```

#
=====
=====
# CONFIGURACIÓN DEL ENTORNO LOCAL PARA META
QUEST
#
=====
=====
# Dirección IP fija asignada en el DNS de tu
router TP-Link
IP_ROUTER = "192.168.0.100"
PUERTO_SERVICIO = 8443

def generar_certificado_dns_fijo():
    """Genera certificados SSL locales con
    soporte SAN correcto para Chromium."""
    cert_file = "dns_local.crt"
    key_file = "dns_local.key"

    if not os.path.exists(cert_file) or not
os.path.exists(key_file):
        print(f"[INFO] Generando certificado
SSL robusto para la IP: {IP_ROUTER}...")

        # 1. Generar la llave privada RSA
        private_key =
rsa.generate_private_key(
            public_exponent=65537,
key_size=2048
        )

        # 2. Configurar los detalles del
emisor (Subject/Issuer)

```

```
        subject = issuer = x509.Name([
x509.NameAttribute(NameID.COUNTRY_NAME, "MX"),
x509.NameAttribute(NameID.STATE_OR_PROVINCE_NAME, "Qro"),
x509.NameAttribute(NameID.ORGANIZATION_NAME,
"Universidad"),
x509.NameAttribute(NameID.COMMON_NAME,
IP_ROUTER),
    ])
```

```
    # 3. PASO CRÍTICO: Configurar
    extensiones SAN (Subject Alternative Name)
```

```
    # Separación estricta exigida por
    Chromium: Dominios en DNSName e IPs en
    IPAddress
```

```
        alt_names = [
            x509.DNSName("localhost"),
            x509.DNSName("127.0.0.1"),

x509.IPAddress(ipaddress.ip_address(IP_ROUTER)
) # La IP va estrictamente aquí como objeto
IPAddress
    ]
```

```
    # 4. Construir el certificado válido
    por 1 año
```

```
        cert = (
            x509.CertificateBuilder()
            .subject_name(subject)
```

```

        .issuer_name(issuer)
        .public_key(private_key.public_key
    ())
        .serial_number(x509.random_serial_
number())
        .not_valid_before(datetime.datetime
e.utcnow())
        .not_valid_after(
            datetime.datetime.utcnow() +
datetime.timedelta(days=365)
        )
        .add_extension(
x509.SubjectAlternativeName(alt_names),
critical=False
        )
        .add_extension(
x509.ExtendedKeyUsage([x509.oid.ExtendedKeyUsa
geOID.SERVER_AUTH]),
            critical=False,
        )
        .sign(private_key,
hashes.SHA256())
    )

    # 5. Guardar la llave privada en el
disco duro (.key)
    with open(key_file, "wb") as f:
        f.write(
            private_key.private_key_bytes(
encoding=serialization.Encoding.PEM,

```

```

format=serialization.PrivateFormat.TraditionalOpenSSL,

encryption_algorithm=serialization.NoEncryption(),

    )

    )

        # 6. Guardar el certificado en el disco duro (.crt)
        with open(cert_file, "wb") as f:

f.write(cert.public_bytes(serialization.Encoding.PEM))

        print("[OK] Certificados SSL creados con éxito.")

    return cert_file, key_file

#
=====
=====
# CONFIGURACIÓN DE FASTAPI (IDÉNTICA A TU VERSIÓN DE CELULARES)
#
=====
=====
app = FastAPI()

app.add_middleware(
    CORSMiddleware,

```

```
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

# Monta la raíz de la carpeta actual de manera
# idéntica a la solución http de los teléfonos
app.mount("/", StaticFiles(directory=".",
html=True), name="static")

if __name__ == "__main__":
    # Obtener las rutas de los certificados
    seguros
    cert_path, key_path =
    generar_certificado_dns_fijo()

    print(f"\n[SERVIDOR] Iniciando entorno
    seguro para Meta Quest en:
    https://{IP_ROUTER}:{PUERTO_SERVICIO}")
    print("[INSTRUCCIÓN] En el casco abre Meta
    Quest Browser e ingresa exactamente la URL de
    arriba.\n")

    # Arrancamos Uvicorn forzando HTTPS nativo
    a través de la red local inalámbrica
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=PUERTO_SERVICIO,
        ssl_keyfile=key_path,
        ssl_certfile=cert_path,
        reload=True
```

)

Nota técnica antes de la clase:

Al arrancar este script por primera vez en la Alienware, se generarán automáticamente los archivos `dns_local.crt` y `dns_local.key` en la raíz de `ClaseWebXRMeta`. Esto significa que la criptografía se aplicó con éxito.

Paso 3: El Código de la Escena en Three.js (Requisitos de Realidad Mixta)

Para evitar bloqueos y asegurar que los personajes se proyecten fluidamente sobre el entorno real (Passthrough) eliminando los dientes de sierra, verificar que los archivos de código JavaScript de los proyectos incluyan estos tres parámetros obligatorios:

1. Optimizar el renderizador (WebGLRenderer): Activa el suavizado de bordes y la transparencia para el fondo.

```
const renderer = new THREE.WebGLRenderer({
  antialias: true, //
  Elimina por completo los dientes de sierra
  alpha: true, //
  PERMITE VER LA SALA REAL EN EL FONDO
  (Passthrough)
  powerPreference: "high-performance" //
  Fuerza el uso de la GPU móvil a su máximo
  rendimiento
});
document.body.appendChild(renderer.domElement);
renderer.xr.enabled = true; //
Habilita el soporte nativo de WebXR
```

2. Utilizar el botón de AR (Realidad Mixta): Sustituir cualquier botón de VR por el componente de Realidad Aumentada.

```
import { ARButton } from  
'three/addons/webxr/ARButton.js';  
document.body.appendChild(ARButton.createB  
utton(renderer));
```

3. Bucle de animación compatible: Reemplaza `requestAnimationFrame` por el ciclo interno de WebXR para que la imagen inalámbrica no se congele al pulsar el botón.

```
renderer.setAnimationLoop(function () {  
    // Aquí adentro se actualiza la lógica  
    de los personajes y animaciones  
    renderer.render(scene, camera);  
});
```

Paso 4: Flujo de Ejecución en el Salón de Clases (Sin Cuenta de Desarrollador)

Seguir estrictamente este orden durante las sesiones académicas para conectar el visor de forma inalámbrica a la velocidad de la red local:

1. Enciende el Router: Conecta el TP-Link Gamer a la corriente (no se requiere el internet de la universidad).
2. Sincroniza la Alienware: Conecta la laptop al Wi-Fi emitido por el router TP-Link.
3. Sincroniza el Casco: Colocarse las Meta Quest 3 y conectarlas a la misma red Wi-Fi del router.

4. Arranca el Servidor: Abrir el CMD, navegar a la carpeta correspondiente y ejecuta el script:

```
cd "D:\Diseno Interactivo 2 Sem 2025\  
ClaseWebXRMeta"  
python main.py
```

5. Navegar desde el Visor: Abrir la aplicación nativa Meta Quest Browser (el navegador estándar de las gafas).
6. Ingresar la Dirección URL: En la barra superior escribir la dirección exacta del servidor:
`https://192.168.0.100:8443`
7. Saltar la advertencia de seguridad: El navegador mostrará una pantalla indicando "*La conexión no es privada*". Esto ocurre porque el certificado es autofirmado por el usuario. Hacer clic en el botón Opciones Avanzadas (Advanced) en la esquina inferior izquierda y luego selecciona Proceder a la dirección (no seguro) o Proceed anyway.
8. ¡Listo para usar! La página del index cargará de forma inmediata. Al acceder al proyecto de cualquier equipo, el botón de Realidad Mixta (AR) estará activo. Al pulsarlo, los alumnos verán los elementos 3D perfectamente integrados en el salón con total nitidez.

Código Definitivo para el `index.html` de los Proyectos

Este es el código que se debe distribuir a los alumnos (o aplicar en sus carpetas dentro de

`ClaseWebXRMeta/EquipoXX/proyXX/index.html`):

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>three.js webgl - equirectangular video panorama</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, user-
scalable=no, minimum-scale=1.0, maximum-scale=1.0">
<link type="text/css" rel="stylesheet" href="main.css">
<style>
body {
touch-action: none;
}
</style>
</head>
<body>
<!-- Import maps polyfill -->
<!-- Remove this when import maps will be widely supported -->
<script async src="https://unpkg.com"></script>
<script type="importmap">
{
"imports": {
"three": "./build/three.module.js",
"three/addons/": "./jsm/"
}
}
</script>
```

```
<script type="module">
import * as THREE from 'three';
import { ARButton } from 'three/addons/webxr/ARButton.js';
//Add this for stats animation
import Stats from 'three/addons/libs/stats.module.js';
import { FBXLoader } from 'three/addons/loaders/FBXLoader.js';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

let camera, scene, renderer, stats;
let controller;
//Add clock
const clock = new THREE.Clock();
//Add Mixer
let mixer;

// CORRECCIÓN CRÍTICA: Se eliminó la llamada directa a init() y
animate() aquí
// para evitar el desbordamiento de memoria gráfica y congelamiento
en el Quest 3.
init();

function init() {
    const container = document.createElement( 'div' );
    document.body.appendChild( container );
    scene = new THREE.Scene();

    camera = new THREE.PerspectiveCamera( 70,
window.innerWidth / window.innerHeight, 0.01, 20 );

    scene.add( new THREE.HemisphereLight( 0xa5a5a5, 0x898989,
3 ) );
    const light = new THREE.DirectionalLight( 0xffffff, 3 );
```

```
light.position.set( 1, 1, 1 ).normalize();
scene.add( light );
```

```
//Load GLTF Model
new GLTFLoader()
.setPath( 'models/gltf/' )
.load( 'Nave.gltf', function ( gltf ) {
    gltf.scene.scale.set(.003, .003, .003);
    gltf.scene.position.x = -0.8;
    gltf.scene.position.y = 0.3;
    gltf.scene.position.z = -1.5;
    scene.add( gltf.scene );
} );
```

```
// Load FBX Model and Animation
const loader = new FBXLoader();
loader.load( 'models/fbx/Lenador.fbx', function ( object ) {
    // Add scale and transform
    object.scale.set(.004, .004, .004);
    object.position.x = 0.8;
    object.position.y = -0.5;
    object.position.z = -1.5;
```

```
    mixer = new THREE.AnimationMixer( object );
    const action = mixer.clipAction( object.animations[ 4] );
    action.play();
```

```
    object.traverse( function ( child ) {
        if ( child.isMesh ) {
            child.castShadow = true;
            child.receiveShadow = true;
        }
    } );
```

```
    } );  
    scene.add( object );  
  } );
```

```
  // MODIFICACIÓN OPTIMIZADA: Configuración del  
  renderizador forzando la GPU nativa del Quest 3  
  renderer = new THREE.WebGLRenderer( {  
    antialias: true,  
    alpha: true,  
    powerPreference: "high-performance" // Fuerza el rendimiento  
    máximo inalámbrico
```

```
  } );  
  renderer.setPixelRatio( window.devicePixelRatio );  
  renderer.setSize( window.innerWidth, window.innerHeight );  
  renderer.xr.enabled = true;  
  container.appendChild( renderer.domElement );
```

```
  // ADD stats  
  stats = new Stats();  
  container.appendChild( stats.dom );
```

```
  // Creación del botón nativo de Realidad Mixta (Passthrough)
```

```
document.body.appendChild( ARButton.createButton( renderer ) );
```

```
  window.addEventListener( 'resize', onWindowResize );  
}
```

```
function onWindowResize() {  
  camera.aspect = window.innerWidth / window.innerHeight;  
  camera.updateProjectionMatrix();  
  renderer.setSize( window.innerWidth, window.innerHeight );
```

```

}

// Bucle de animación estructurado para WebXR
function animate() {
    const delta = clock.getDelta();
    if ( mixer ) mixer.update( delta );
    stats.update();
    render();
}

// CORRECCIÓN CRÍTICA: Único disparador y encargado de
controlar el ciclo de renderizado en el casco
renderer.setAnimationLoop( animate );

function render() {
    renderer.render( scene, camera );
}
</script>
</body>
</html>

```

¡Adiós a los dientes de sierra! Al ejecutar esta solución, los bordes de los modelos 3D se verán completamente pulidos y definidos en el visor.

El aliasing (los dientes de sierra) que se veían antes ocurría por una mala compresión de video provocada por el cable y el conflicto de las tarjetas gráficas en la Alienware. Con este nuevo enfoque, se eliminan de raíz gracias a tres factores técnicos:

- Antialias nativo: La propiedad `antialias: true` que ya tienen en el código en el `index.html` ahora sí se activará de verdad. Antes, el navegador la ignoraba por el cuello de botella del cable Link.
- Procesamiento inalámbrico autónomo: Al no usar Meta Horizon Link, la Alienware ya no procesa los gráficos; ahora es el potente chip Snapdragon XR2 Gen 2 dentro del Meta Quest 3 el que renderiza la escena directamente frente a los ojos de los usuarios, aplicando sus propios filtros de suavizado de bordes en tiempo real.
- Máximo rendimiento forzado: La instrucción `powerPreference: "high-performance"` que se agregó, se le ordena al procesador del casco que use toda su potencia para suavizar las líneas geométricas del leñador y la nave.

Se verá la escena con una buena nitidez, idéntica a la de una aplicación nativa instalada en el casco.

Se puede mantener el casco conectado a la corriente usando tu cargador Anker de 65W y el cable Silkland de 3 metros mientras prosigue la clase inalámbrica.

De hecho, esta combinación es una buena estrategia para tu laboratorio local por tres razones técnicas:

1. Carga rápida real (Power Delivery): El cargador Anker de 65W tiene potencia de sobra para alimentar las Meta Quest 3, y el cable Silkland de 240W cuenta con el chip electrónico

necesario para negociar el voltaje correcto. Esto garantiza que la batería del casco se mantenga al 100% o incluso se cargue mientras se está usando.

2. Cero conflictos gráficos: Al conectar el cable Silkland directamente al cargador Anker en la pared (o en un contacto del salón) en lugar de conectarlo a la laptop, el cable solo transportará energía eléctrica. La Alienware m16 R1 queda **completamente libre y el software Meta Horizon Link no se activará, evitando de raíz los conflictos de tarjetas gráficas, los dientes de sierra y los bloqueos de ayer.**
3. Libertad de movimiento de 3 metros: Al ser un cable de 3 metros, proporcionará el radio perfecto para moverse alrededor del escritorio del instructor, ponerse el casco, interactuar con la escena de Three.js

El laboratorio queda configurado de forma perfecta: la Alienware emite el servidor seguro por el Wi-Fi del router TP-Link, el casco procesa todo de forma inalámbrica y el combo Anker/Silkland le da energía infinita para durar todas las horas de clase que se necesiten.